

Interview Deep Dive: The Database Layer (`db/schema.js`)

If an interviewer asks you to explain your database design, this is your moment to prove you understand **data integrity**, **performance at scale**, and **relational modeling**.

Here is exactly how to break down your `schema.js` file with absolute confidence.

1. The Architectural Choice: Why Postgres & Drizzle?

Start by justifying your tooling before looking at the code.

You say:

"For the data layer, I chose PostgreSQL. Because FollowUpHub handles tasks, escalations, and a chronological audit trail, I needed strict relational integrity and ACID compliance, which a NoSQL database like MongoDB wouldn't easily provide.

To manage the schema, I deliberately chose **Drizzle ORM** over heavier alternatives like Prisma or Sequelize. I love Drizzle because it's incredibly lightweight, has zero cold-start latency, and the schema definitions map almost exactly 1-to-1 with raw SQL while still giving me 100% TypeScript type safety. Let me show you what I mean in `schema.js`."

2. Enums: Strict State Management

Show them how you prevent bad data from ever entering the DB.

You say:

"If you look at the top of the file, the very first thing I define are my PostgreSQL Enums—like `followupStatusEnum` and `reminderPolicyEnum`.

I didn't just enforce these states in my Express controllers; I enforced them at the database level. This guarantees that a follow-up can *only* ever be `PENDING`, `SNOOZED`, `ESCALATED`, or `DONE`. It prevents random strings from ever polluting the database and makes the application mathematically predictable."

3. The Core Architecture: Users & Follow-ups

Walk through the main entity relationship.

You say:

"Moving down, we have the `usersTable` and the `followups` table.

In `followups`, notice how I'm handling time: every timestamp uses `timestamp(..., { withTimezone: true })`. This is absolutely critical for a productivity app. Because users could be in different time zones, storing due dates with timezone awareness ensures the cron engine always fires exactly when the user expects it to.

I also track state transitions explicitly. Rather than just relying on an `updatedAt` field, I specifically track `completedAt`, `lastReminderSentAt`, and `lastManagerNotifiedAt`. This gives the reminder engine the exact data it needs to calculate cooldowns."

4. The Append-Only Audit Trail

Interviewers love event-driven/audit patterns.

You say:

"One of my favorite design decisions is the `followup_events` table.

Rather than just updating the status of a follow-up in place, every time an action occurs—like a reminder is sent or a task is snoozed—I insert a new row into the `followup_events` table.

This is an **append-only audit trail**. It allows me to render a full historical timeline on the frontend so the user can see exactly when a task was created, how many times they snoozed it, and when it escalated. It turns the app from a simple to-do list into a robust tracking system."

5. Performance Optimization: Indices (The "Wow" Factor)

This is the most senior concept in this file. Nail this explanation.

You say:

"Finally, let's talk about performance. At the bottom of every single table definition, you'll see a second argument where I define **Indexes**.

For example, on the `followups` table, I didn't just index the primary key. I explicitly created a `dueAtIdx` and a `statusIdx`.

Why? Because my backend cron job runs every 60 seconds and executes a query that basically says: *'Find all follow-ups where status is PENDING and dueAt is in the past.'* If I didn't index those columns, PostgreSQL would have to do a full table scan every single minute.

By indexing `status` and `dueAt`, that cron job query executes in milliseconds, even if there are a million tasks in the database.

I applied this same logic everywhere: indexing `userId` across all tables because almost every query is scoped to a specific user."

Why Interviewers Will Love This Answer:

By explaining your schema this way, you prove:

1. **You understand time zones:** (`withTimezone: true` is a classic senior dev trap).
2. **You understand data history:** (Append-only event tables show maturity).
3. **You understand performance:** (Knowing *why* you indexed `dueAt` for the cron job proves you think about how your app will behave with 100,000 users, not just 10).